

Assignment 4: Monads, Type Inference, and Subtyping

CSC324H1, Fall 2025

Due Date: November 21, 2025 (11:59 pm)

In this assignment, you will:

- Use monads to program in an imperative style in Haskell;
- Implement a type inference algorithm for the simple language we have been working on;
- Add support for subtyping to the typechecker implemented in the previous assignment.

Please read the instructions carefully before you start.

Instructions

For Part 1, you have been provided with starter code in `Imperative.hs`. For Parts 2 and 3, a starter project is provided to you. You should complete the functions as required later in the handout and marked with `error "TODO"` in the code. Run the tests provided to you to check your work. Note that your code will be graded based on additional hidden tests.

Please note that for all assignments:

- It is recommended to only submit all required files as specified on MarkUs.
- You are responsible for making sure that your code has no syntax errors or compile *errors*. If your file cannot be imported by another file, you may receive a grade of zero. Compiler *warnings* can help you locate potential issues, but they do not affect your grade.
- If you're not intending to complete one of the functions, do *not* remove the function signature. If you are including a partial solution, make sure it doesn't cause a compile error. If your partial solution causes compile errors, it's better to comment out your solution (but not the signature).
- Do not change the `module ... where` and `import ...` lines of your code. We may not be able to mark your submission if these lines are modified.

Part 1: Imperative Functional Programming

30 pts

In this part, you will define `def`, `var`, `lit`, `while`, `+=`, `-=` and `*=` functions to program in an imperative style in Haskell with the State monad in `Imperative.hs`. One example of such program is provided in the starter code as function `factorial`.

There are two kinds of values in this minimal imperative language: variables and literals. We can define a `Value` type that represents either a variable represented by an integer index or an integer literal.

```
data Value = Var Int | Lit Int
```

Recall how we defined the environment-based operational semantics of lambda calculus: we had an environment that mapped variables to values. Similarly, here we need a context that maps variable indices to values, and we update it when variables are created and modified:

```
type Ctx = [(Int, Int)]
```

(a) Define `lit` and `var`

6 pts

Implement the following functions:

- `lit` that creates a `Value` from an integer literal.
- `var` that creates a new variable in the context with the integer argument as its initial value, and returns a `Value` representing that variable.

You may need to update the state with `put` to keep track of the new variable. Use `do` notation to make it easier to work with the State monad.

(b) Define `def`

3 pts

`def` is a helper function that takes an imperative program, runs it with an initial state, and returns the final result.

(c) Define `+=`, `-=`, and `*=`

9 pts

`+=`, `-=`, and `*=` functions should modify the value of a variable in the context, and return `()`. If the first argument is a `Lit`, then they do nothing.

(d) Define `while`

3 pts

`while` takes a value, a predicate function, and a body program. It runs the body program repeatedly as long as the predicate function returns `True` on the value:

$$\text{while cond pred body} = \begin{cases} \text{body; while cond pred body} & \text{if (pred cond)} \\ () & \text{if not (pred cond)} \end{cases}$$

(e) Correctness of Imperative Style Code

9 pts

Make sure that the provided imperative function `factorial` behaves correctly. Your implementation should also be able to correctly handle other programs defined in a similar style.

Part 2: Type Inference

50 pts

In this part, you will implement type inference for the simply-typed lambda calculus with booleans and natural numbers using constraint generation and unification.

(a) Types and Terms

The types (defined in `Type.hs`) concerned in this part include booleans, natural numbers, and function types. Additionally, we include type variables for inference and a helper function to get the free type variables of a type:

```
data Type = TyBool
          | TyNat
          | TyArrow Type Type
          | TyVar TypeVar    -- meta type variables for inference
          ... -- other cases are irrelevant to this part

-- Type variable names are represented by integers
data TypeVar = TypeVar Int

-- Get the free type variables in a type
ftv :: Type -> [TypeVar]
```

We use the nameless representation of terms (`Term` in `Term.hs`) as in previous assignments. The relevant fragment of the term syntax for this part is:

```
data Term = TmTrue | TmFalse
          | TmIf Term Term Term
          | TmZero | TmSucc Term | TmPred Term | TmIsZero Term
          | TmVar Int
          | TmApp Term Term
          | TmAbs Term          -- unannotated lambda
          | TmAnnAbs Type Term  -- annotated lambda
          ... -- other cases are irrelevant to this part
```

(b) Substitutions

Substitutions are used to represent the mapping from type variables to types. The data type and functions for substitutions are defined in `Type.hs` as follows:

```
-- Substitutions for type variables
data Subst = Subst (Map TypeVar Type)

emptySubst :: Subst
composeSubst :: Subst -> Subst -> Subst
substType :: Subst -> Type -> Type
```

Name	Note
Subst	Substitutions, as a mapping from type variables to types.
emptySubst	The empty substitution.
composeSubst	Composes two substitutions.
substType	Substitutes a type with a substitution.

We use Haskell’s built-in [Map](#) data type to implement substitutions. Useful map operations are imported in the starter code. You may refer to [the documentation](#) for details and examples.

(c) Monad for Constraint Generation

Instead of non-deterministic choices of fresh type variable names in the constraint-based typing, we use a monad for constraint generation (in `InferMonad.hs`) to generate fresh type variables by keeping track of a counter state. It also allows us to signal errors:

```

type FreshId = Int

data InferM a = InferM (FreshId -> InferResult a)
data InferResult a
    = InferOk a FreshId
    | InferError String

instance Functor InferM where ...
instance Monad InferM where ...
-- You may ignore the Applicative instance

runInferM      :: InferM a -> InferResult a
freshId        :: InferM FreshId
freshTypeVar   :: InferM TypeVar
inferFail      :: String -> InferM a

```

Name	Note
FreshId	The type for the next fresh type variable id.
InferM and its type class instances	The inference monad for constraint generation.
InferResult	The result of the constraint generation.
InferOk	Indicates operation within the monad returned successfully with the result and the next fresh type variable id.
InferError	Indicates a failure with an error message.
runInferM	Runs the inference monad.
freshId	Returns the next fresh type variable id.
freshTypeVar	Generates a fresh type variable.
inferFail	Signals an error with an error message.

(d) Constraint Generation

We generate equality constraints between types to be solved later. The data type for constraints is defined in `Type.hs`:

```
data Constraint = CEq Type Type
type Constraints = [Constraint]
```

The constraint generation function is declared in `Infer.hs` as follows:

```
type InferContext = [Type]
infer :: InferContext -> Term -> InferM (Type, Constraints)
```

`infer` takes a typing context and a term, and returns the type inference monad that contains the inferred type and the constraints to be solved, or an error message if the constraint generation fails.

`InferContext` is defined similarly as the typing context in the previous assignment, which is also a list of types indexed by the de Bruijn indices of term variables.

(e) Unification

Constraints generated by `infer` are solved by unification with occurs check. Inside `Unify.hs`, we declare:

```
data UnifyResult a = UnifyOk a | UnifyError String

instance Functor UnifyResult where ...
instance Monad UnifyResult where ...
```

```
unifyConstraints :: Constraints -> UnifyResult Subst
```

`UnifyResult` is a wrapper over the result of unification. We implement it as a monad to allow for convenient chaining of operations. `unifyConstraints` takes a list of constraints and returns a substitution when all constraints are solved successfully inside `UnifyResult`, or an error message otherwise.

Finally, after generating constraints and solving them, applying the resulting substitution to the inferred type gives the result of type inference, which is either the inferred type or an error message:

```
-- Inside Infer.hs
inferType :: Term -> Either String Type
```

Your Tasks and Tips

- (15 pts)** Implement `substType`, `composeSubst`, and `ftv` in `Type.hs`.
 - You should essentially implement the substitution application and composition described in Section 22.1.1 of TAPL.
 - Your specific handling of duplication of type variables in `ftv` or the internal structure of the substitution mappings do not affect marking.
- (0 pts)** Implement instances of `Functor` and `Monad` for `InferM` in `InferMonad.hs`, and function `freshId` for generating fresh type variable ids.
 - The inference monad is essentially a combination of the state monad and the error (or `Either`) monad. You may refer to the lecture and tutorial materials to further understand this.

- The `Applicative` instance is provided and you do not need to modify it.
3. **(0 pts)** Implement instances of `Functor` and `Monad` for `UnifyResult` in `Unify.hs`.
 - There is no point assigned to both Tasks 2 and 3 and no test cases are provided. You should make sure that your implementation is correct and satisfies the monadic laws to ensure correct behavior of your program.
 4. **(10 pts)** Implement the constraint generator `infer` in `Infer.hs`.
 - Your constraint generator should essentially implement the constraint typing rules as shown in Figure 22-1 and Section 22.6 (for unannotated lambda) of TAPL.
 - You may ignore all the cases of `Term` other than the ones listed in (a) in this part of the handout. We will not test on these cases.
 5. **(10 pts)** Implement `unifyConstraints` in `Unify.hs`.
 - You should essentially implement the unification algorithm described in Figure 22-2 of TAPL.
 6. **(15 pts)** Make sure that `inferType` in `Infer.hs` returns a correct type or an error message when the term is not well-typed.
 - Testing for this task will compare the inferred type with the expected type after proper renaming of type variables, which means your specific implementation of type variable numbering does not affect marking. For example:
`TyArrow (TypeVar (TyVar 0)) (TyArrow (TypeVar (TyVar 1)) (TypeVar (TyVar 0)))`
 is considered equivalent to
`TyArrow (TypeVar (TyVar 1)) (TyArrow (TypeVar (TyVar 2)) (TypeVar (TyVar 1)))`
 Error message texts are not graded.
 - Tests provided in the starter code are for the correctness of the whole type inference process. Marks for each component (i.e. points assigned to Tasks 4 and 5) will be given as well.

Part 3: Typechecking with Subtyping

20 pts

In this part, you will get back to the typechecker implemented in the previous assignment and modify it to support subtyping.

(a) The Top Type `TyTop`

We introduce the top type to the type syntax in `Type.hs`:

```
data Type = TyBool | TyNat | TyTop
          | TyArrow Type Type
          | TyRecord [(String, Type)]
          | TyProd Type Type
          | TySum Type Type
          ...      -- type variables are irrelevant to this part
```

(b) Subtyping

We declare `(<:)` as an algorithmic implementation of subtyping in `TypeCheck.hs`, which takes two types and returns a boolean indicating whether the first type is a subtype of the second type:

```
(<:) :: Type -> Type -> Bool
```

The typechecker `typeCheck` in `TypeCheck.hs` should call `(<:)` at appropriate places.

Your Tasks and Tips

1. **(15 pts)** Implement `(<:)` in `TypeCheck.hs`.
 - You should essentially implement the algorithmic subtyping in Figure 16-2 of TAPL and extend the rules to support subtyping for the other cases.
 - You may ignore subtyping for type variables. We will not test on that case.
2. **(5 pts)** Modify the typechecker `typeCheck` in the starter code to support subtyping.
 - Your typechecker should support the algorithmic typing rules in Figure 16-3 of TAPL and the extensions for other language constructs.
 - For conditional and match expressions, we require both branches to have the same type.
 - You may find textbook Chapter 17 useful.
 - You may ignore the cases of `Term` for unannotated lambdas `TmAbs` for the typechecker. We will not test your typechecker on this case. Error message texts are also not tested.